

Predicting Bazel Build Times: Reproduction and Improvement

UE24CS352A Machine Learning Mini-Project | Team: [Name 1, SRN] and [Name 2, SRN] | Code: [GitHub repo link]

1. Introduction and Problem

Bazel is a build tool that compiles and tests large code bases. Builds can take long and use a lot of CPU. If we know in advance how much CPU time a build will need, developers can plan for it. We take the paper *Predicting Bazel Build Times Using Machine Learning* (Nadeem and Raza, Stanford CS229) and reproduce its approach: predict the **CPU time (ms) of a build** from the files that changed. We then check the original implementation, fix what stops a valid evaluation, and test one correction. This is a reproduction and improvement of the paper's pipeline, not a new algorithm.

2. Dataset

We used the data in the paper's own repository (nothing new was collected): 500 builds of the open-source *bazelbuild/bazel* project. For each build we have the most common **path prefix** of the changed files (8 values, e.g. *src/main*, *site/*), the most common **file type** (4 values: JAVA, C/C++, Starlark, python) and the **CPU time** in ms. CPU time is very skewed: median about 1.8 s, mean about 4.1 s, max 37 s, and about 30% of builds are near no-op rebuilds (about 0.6 s). We checked that the CSV values match the 500 raw build-event JSON files. Build 1 is a cold-start build (44.9 s) and was dropped.

3. Methodology

Features and target. Inputs are *prefix*, *type* and the paper's **feature cross** (prefix|type), all one-hot encoded. Target is CPU time. Nothing measured after the build (CPU time, duration, action counts) is used as input, so there is no target leakage. **Split.** Contiguous blocks in build order: train 299, validation 100, test 100, with no overlap. **Model.** Linear regression with L2 regularisation (Ridge, scikit-learn). Only the regularisation strength was tuned, on validation RMSE. The test set was read only once, in the final evaluation script. **Metrics.** RMSE, MAE and R^2 in milliseconds.

4. Issues in the original implementation

(a) **Untrained test model.** The original *test_model()* builds a new regressor and never trains it, so it predicts 0. Predicting 0 gives RMSE 7,459 ms on the test block, which matches the flat line of about 7,500 ms in the paper's test plots. (b) The validation slice *.loc[301:400]* is inclusive, so one row is in both validation and test. (c) The grid search described in the paper is not in the code. (d) **Features do not match the target.** The script runs *git checkout HEAD~1*, saves the diff of that commit, and then runs an *incremental* build. The build therefore has to rebuild the change set recorded in the **previous** row. With the original pairing, the features explain no more variance than shuffled labels (3.6%, while shuffled labels reach up to 8.4%). With the previous row's features they explain 28.0%. This explanation comes from reading the script and the data; we could not re-run the original machine. (e) The extractor matches substrings (.c matches .css), so 20 of 60 *site/* builds are labelled C/C++. We cannot fix this because the raw diffs are not in the repo.

5. Baseline and improvement

Model A (clean baseline): the paper's features, pairing and feature cross, with a proper train / validation / test evaluation. It is not an exact copy: we use scikit-learn Ridge instead of TensorFlow SGD. **Model B (corrected pipeline):** exactly the same, except each build is paired with the features of the change set that was actually rebuilt (the previous row). Only this pairing changes. On validation only, B was clearly better (RMSE 4,264 vs 5,518), so B was chosen as the final approach before the test set was opened. A log-transformed target was tried on validation and was worse, so it was dropped.

6. Results (test set, 100 builds)

Model	Val RMSE	Test RMSE (ms)	Test MAE (ms)	Test R^2
Untrained model (predicts 0), as in paper's <i>test_model()</i>		7459	4707	-0.662
Mean predictor		5821	4299	-0.012
Model A: paper baseline	5518	5818	4299	-0.011
Model B: corrected alignment	4264	5354	3645	0.144

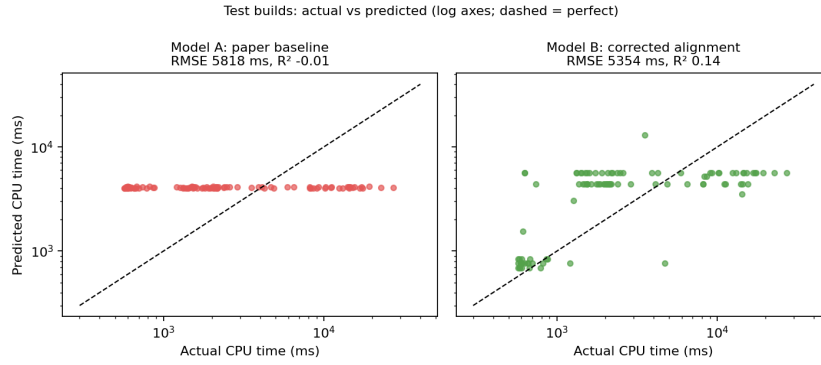


Figure 1: actual vs predicted CPU time on the test builds (log axes). Model A predicts almost a constant; Model B separates cheap builds from expensive ones.

7. Discussion

Model B is better on all three metrics. A paired bootstrap over the 100 test builds gives an RMSE gain of 464 ms (95% interval 185 to 698), so the gain is unlikely to be luck, but it is **modest**: R^2 is 0.14 on test, well below 0.40 on validation. The gain comes mostly from cheap builds (under 1.5 s): MAE 1,312 ms for B vs 3,230 ms for A. B learns that changes to *site/* or *src/test* give almost no-op rebuilds. For the 62 expensive builds B is not better (MAE 5,074 vs 4,953 ms), because prefix and type cannot tell how big a *src/main* rebuild will be.

8. Limitations and Conclusion

Limitations: only 500 builds from one project and one machine; only 100 test builds with a heavy tail, so the estimate is noisy; the features are coarse (no file counts or dependency information); incremental cache state is not modelled; the labelling artefact in the data cannot be repaired; and the alignment cause is inferred, not re-measured.

Conclusion: the paper's reported test results do not measure a trained model, and its features were paired with the wrong builds. After fixing the evaluation and the pairing, a simple linear model with feature cross gives a real but modest improvement over the baseline (test RMSE 5,354 vs 5,818 ms; R^2 0.14 vs -0.01). Better features such as the number of changed files and dependency information, and more builds, are the next steps.